

Unit 1

Basics of 'C' Programming

1.1 Fundamentals of algorithms: Notion of algorithm, Notations used for assignment statements and basic control structure:

Notion of Algorithm:

The study of algorithms is the cornerstone of computer science. It can be recognized as the core of computer science. Computer programs would not exist without algorithms. With computers becoming an essential part of our professional & personal life's, studying algorithms becomes a necessity, more so for computer science engineers. Another reason for studying algorithms is that if we know a standard set of important algorithms, they further our analytical skills & help us in developing new algorithms for required applications

Notations used for assignment statements and basic control structure:

1. Algorithm Name
2. Comments
3. I/P and O/P
4. Steps
5. Variables
6. Assignment Statement
7. If statement/ conditional statement
8. Repeat Statement
9. End of Algorithm.

- 1) **Algorithm Name:** Each algorithm should have name so that user/programmers will know for which problem the algorithm is. So the header contains the name of that algorithm.
- 2) **Comments:** The contain followed by \\ symbol are considered as comments. Comments are necessary to provide extra information about the Statement.
- 3) **I/P and O/P:** I/P and expected O/P of an algorithm should be provided.
- 4) **Steps:** Provide number to all the steps in an algorithm.
- 5) **Variables:** All variable name should be written in upper case

- 6) **Assignment Statement:** For Assignment use $\leftarrow$ symbol to assign the value of right hand side to left hand side. e.g. $A \leftarrow 20$
- 7) **If statement/ conditional statement:** This statement will be written in the following format

If condition1:

Statement1

If condition2:

Statement2

- 8) **Repeat Statement:** This statement is repeated until the condition becomes false. You can create a repeat condition by using the do, while loop.
- 9) **End of Algorithm:** Every algorithm last line should be End/ Stop.

Algorithm:

- It is step by step procedures or a set of instruction for performing a specific task or solving a problem.
- It's used to describe algorithm in a clear, unambiguous and human readable manner.
- An algorithm is a set of rules that specify the order and kind of arithmetic operations that are used on a specified set of data.
- An algorithm is an effective method for solving a problem using a finite sequence of instruction. An algorithm is a finite, definite, effective procedure with some output.

Qualities of good algorithm:

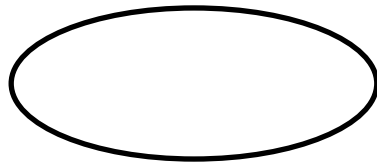
- Input and output should be defined precisely.
- Each step in the algorithm should be clear and unambiguous.
- Algorithm should be most effective among many different ways to solve a problem.
- An algorithm should not include computer code. Instead the algorithm should be written in such a way that can be used in different programming languages.

Flowchart:

- A flowchart is a graphical representation of the algorithm as discussed earlier.
- The flowchart gives a very nice idea of the flow of the algorithm in sequence as well as the conditional changes in the sequence.
- Before drawing flowcharts, we need to know the symbol used for making the flowchart.

Basic Symbols used in Flowchart Designs

1. **Terminal:** The oval symbol indicates Start, Stop and Halt in a program's logic flow. Terminal is the first and last symbols in the flowchart.



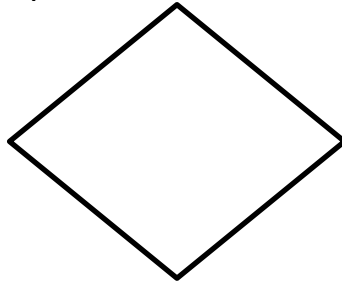
2. **Input/Output:** A parallelogram denotes any function of input/output type. Program instructions that take input from input devices and display output on output devices are indicated with parallelogram in a flowchart.



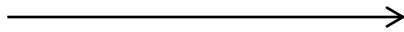
3. **Processing:** A box represents arithmetic instructions. All arithmetic processes such as adding, subtracting, multiplication and division are indicated by action or process symbol.



4. **Decision** Diamond symbol represents a decision point. Decision based operations such as yes/no question or true/false are indicated by diamond in flowchart.



- **Flow lines:** Flow lines indicate the exact sequence in which instructions are executed. Arrows represent the direction of flow of control and relationship among different symbols of flowchart.



Rules for Creating Flowchart:

A flowchart is a graphical representation of an algorithm. It should follow some rules while creating a flowchart.

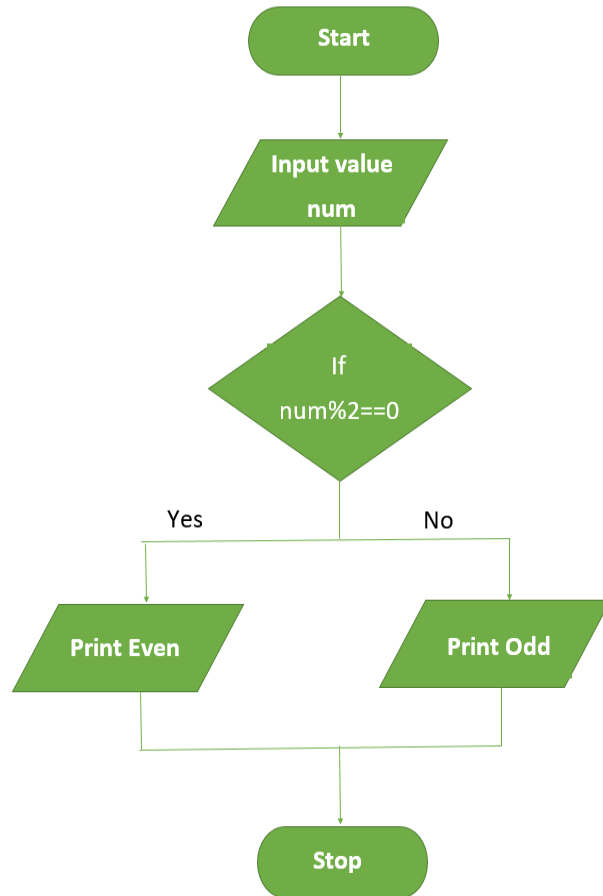
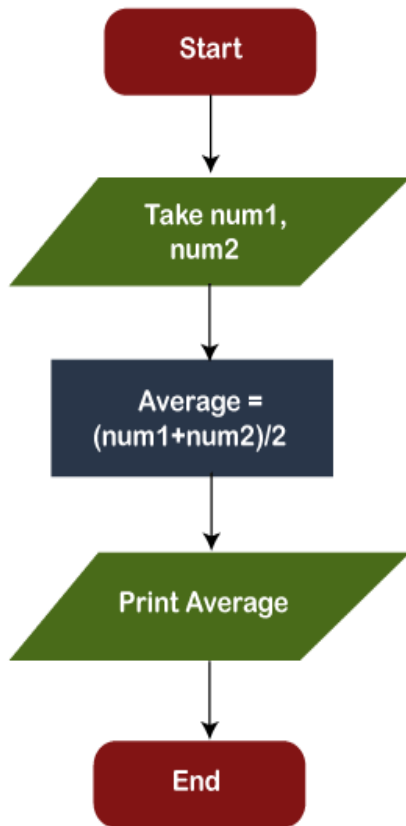
Rule 1: Flowchart opening statement must be 'start' keyword.

Rule 2: Flowchart ending statement must be 'end' keyword.

Rule 3: All symbols in the flowchart must be connected with an arrow line.

Rule 4: The decision symbol in the flowchart is associated with the arrow line.

Example of flowchart:



1.2 Introduction to C

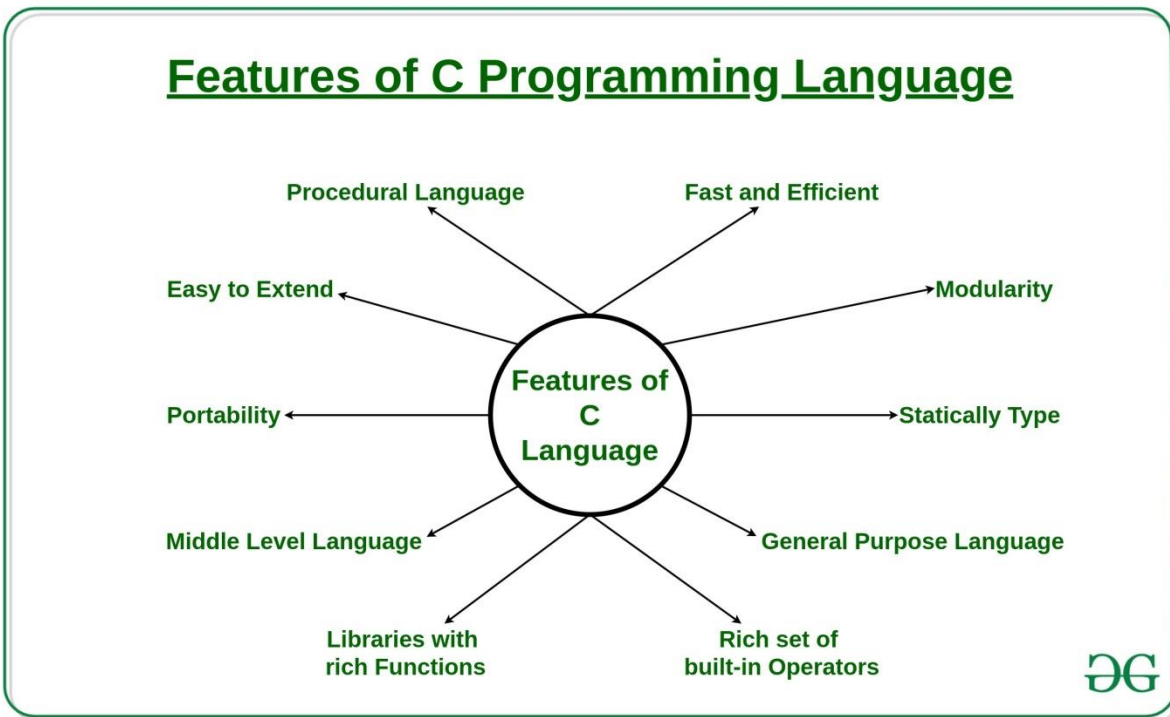
History:

- History of c language is interesting to know.
- C programming language was developed in 1972 by “ Dennis Ritchie” at bell laboratories of AT&T (American Telephone and Telegraph) located in USA.
- Dennis Ritchie is known as the founder of C language.
- It was developed to overcome the problem of previous languages such as B, BPCL,etc.

Most Important Features of C Language

1. Procedural Language
2. Fast and Efficient
3. Modularity
4. Statically Type
5. General-Purpose Language
6. Rich set of built-in Operators

7. Libraries with Rich Functions
8. Middle-Level Language
9. Portability
10. Easy to Extend



1. Procedural Language

In a procedural language like C step by step, predefined instructions are carried out. C program may contain more than one function to perform a particular task. New people to programming will think that this is the only way a particular programming language works. There are other programming paradigms as well in the programming world. Most of the commonly used paradigm is an object-oriented programming language.

2. Fast and Efficient

Newer languages like Java, python offer more features than c programming language but due to additional processing in these languages, their performance rate gets down effectively. C programming language as the middle-level language provides programmers access to direct manipulation with the computer hardware but higher-level languages do not allow this. That's one of the reasons C language is considered the first choice to start learning programming languages. It's fast because statically typed languages are faster than dynamically typed languages.

3. Modularity

The concept of storing C programming language code in the form of libraries for further future uses is known as modularity. This programming language can do very little on its own most of its power is held by its libraries. C language has its own library to solve common problems.

4. Statically Type

C programming language is a statically typed language. Meaning the type of variable is checked at the time of compilation but not at run time. This means each time a programmer types a program they have to mention the type of variables used.

5. General-Purpose Language

From system programming to photo editing software, the C programming language is used in various applications. Some of the common applications where it's used are as follows:

- Operating systems: Windows, Linux, iOS, Android, OXS
- Databases: PostgreSQL, Oracle, MySQL, MS SQL Server, etc.

6. Rich set of built-in Operators

It is a diversified language with a rich set of built-in operators which are used in writing complex or simplified C programs.

7. Libraries with Rich Functions

Robust libraries and functions in C help even a beginner coder to code with ease.

8. Middle-Level Language

As it is a middle-level language so it has the combined form of both capabilities of assembly language and features of the high-level language.

9. Portability

C language is lavishly portable as programs that are written in C language can run and compile on any system with either no or small changes.

10. Easy to Extend

Programs written in C language can be extended means when a program is already written in it then some more features and operations can be added to it.

Structure of the C Program

The basic structure of a C program is divided into 6 parts which makes it easy to read, modify, document, and understand in a particular format. C program must follow the below-mentioned outline in order to successfully compile and execute. Debugging is easier in a well-structured C program.

Sections of the C Program

There are 6 basic sections responsible for the proper execution of a program. Sections are mentioned below:

1. Documentation
2. Preprocessor Section
3. Definition
4. Global Declaration
5. Main() Function
6. Sub Programs

1. Documentation

This section consists of the description of the program, the name of the program, and the creation date and time of the program. It is specified at the start of the program in the form of comments. Documentation can be represented as:

```
// description, name of the program, programmer name, date, time etc.
```

```
or
```

```
/*
```

```
    description, name of the program, programmer name, date, time etc.
```

```
*/
```

Anything written as comments will be treated as documentation of the program and this will not interfere with the given code. Basically, it gives an overview to the reader of the program.

2. Preprocessor Section

All the header files of the program will be declared in the preprocessor section of the program. Header files help us to access other's improved code into our code. A copy of these multiple files is inserted into our program before the process of compilation.

Example:

```
#include<stdio.h>
```

```
#include<math.h>
```

3. Definition

Preprocessors are the programs that process our source code before the process of compilation. There are multiple steps which are involved in the writing and execution of the program. Preprocessor directives start with the '#' symbol. The #define preprocessor is used to create a constant throughout the program. Whenever this name is encountered by the compiler, it is replaced by the actual piece of defined code.

Example:

```
#define long long ll
```

4. Global Declaration

The global declaration section contains global variables, function declaration, and static variables. Variables and functions which are declared in this scope can be used anywhere in the program.

Example:

```
int num = 18;
```

5. Main() Function

Every C program must have a main function. The main() function of the program is written in this section. Operations like declaration and execution are performed inside the curly braces of the main program. The return type of the main() function can be int as well as void too. void() main tells the compiler that the program will not return any value. The int main() tells the compiler that the program will return an integer value.

Example:

```
void main()
```

or

```
int main()
```

6. Sub Programs

User-defined functions are called in this section of the program. The control of the program is shifted to the called function whenever they are called from the main or outside the main() function. These are specified as per the requirements of the programmer.

Example:

```
int sum(int x, int y)
{
    return x+y;
}
```

Structure of C Program with example

Example: Below C program to find the sum of 2 numbers:

```
/ Documentation
/**
 * file: sum.c
 * author: you
 * description: program to find sum.
 */
```

```
#include <stdio.h>
```

```
// Definition
#define X 20
```

```
// Global Declaration
int sum(int y);
```

```
// Main() Function
int main(void)
{
    int y = 55;
    printf("Sum: %d", sum(y));
    return 0;
}
```

```
// Subprogram
int sum(int y)
{
    return y + X;
}
```

Output
Sum: 75

C Header Files

In **C language**, header files contain a set of predefined standard library functions. The **.h** is the extension of the header files in C and we request to use a header file in our program by including it with the C preprocessing directive “**#include**”.

C Header files offer the features like library functions, data types, macros, etc by importing them into the program with the help of a preprocessor directive “**#include**”.

Syntax of Header Files in C

We can include header files in C by using one of the given two syntax whether it is a pre-defined or user-defined header file.

```
#include <filename.h> // for files in system/default directory
```

or

```
#include "filename.h" // for files in same directory as source file
```

The “**#include**” preprocessor directs the compiler that the header file needs to be processed before compilation and includes all the necessary data types and function definitions.

```
#include <stdio.h>
#include "Someheader.h"

int main() {
    -----
    -----
}
```

Header File	Description
<stdio.h>	It is used to perform input and output operations using functions like <u>scanf()</u> , <u>printf()</u> , etc.

<code><math.h></code>	It is used to perform mathematical operations like <code>sqrt()</code> , <code>log2()</code> , <code>pow()</code> , etc.
<code><string.h></code>	It is used to perform various functionalities related to string manipulation like <code>strlen()</code> , <code>strcmp()</code> , <code>strcpy()</code> , <code>size()</code> , etc.
<code><conio.h></code>	It contains some useful console functions.

main() Function

The main function is an integral part of the programming languages such as C, C++, and Java. The **main** function in C is the entry point of a program where the execution of a program starts. It is a user-defined function that is mandatory for the execution of a program because when a C program is executed, the operating system starts executing the statements in the **main()** function.

Syntax

```
void main()
```

```
{
    // Function body
}
```

Important Points about C main Function

- It is the function where the program's execution starts.
- Every program has exactly one main function.
- The name of this function should be “**main**” not anything else.
- The main function always returns an integer value or void.
- The main function is called by OS, not the user.

Example:

```
void main()
{
    // code
    printf("Hello");
}
```

1.3 Fundamental constructs of 'C':

Tokens in C:

A token in C can be defined as the smallest individual element of the C programming language that is meaningful to the compiler. It is the basic component of a C program.

Types of Tokens in C

The tokens of C language can be classified into six types based on the functions they are used to perform. The types of C tokens are as follows:

1. Character Set:

The character set of any programming language indicates the different characters the program can contain.

Character set include all the alphabets, digits and special symbols.

Characters	List included
Alphabets (Upper case and lower case)	A,B,C.....Z & a,b,c.....z
Digits (Numbers)	0,1,2,3.....9
Special symbols (all those seen on a keyboard, nothing behind that)	<>, {}, (), ., :, ;, !, ?, +, "", /, *, %, &, #, \, \$, ^, -,
Other special character	Blank space, tab, enter key

2. Keyword:

The keywords are pre-defined or reserved words in a programming language. Each keyword is meant to perform a specific function in a program. Since keywords are referred names for a compiler, they can't be used as variable names because by doing so, we are trying to assign a new meaning to the keyword which is not allowed. You cannot redefine keywords. However, you can specify the text to be substituted

for keywords before compilation by using C preprocessor directives. **C** language supports **32** keywords and newly one keyword is added which are given below:

auto	double	int	struct	
break	else	long	switch	
case	enum	register	typedef	
char	extern	return	union	
const	float	short	unsigned	
continue	for	signed	void	
default	goto	sizeof	volatile	
do	if	static	while	-packed (Newly added)

3. Identifiers

Identifiers are used as the general terminology for the naming of variables, functions, and arrays. These are user-defined names consisting of an arbitrarily long sequence of letters and digits with either a letter or the underscore(_) as a first character. Identifier names must differ in spelling and case from any keywords. You cannot use keywords as identifiers; they are reserved for special use. Once declared, you can use the identifier in later program statements to refer to the associated value. A special identifier called a statement label can be used in goto statements.

Rules for Naming Identifiers

Certain rules should be followed while naming c identifiers which are as follows:

- They must begin with a letter or underscore(_).
- They must consist of only letters, digits, or underscore. No other special character is allowed.
- It should not be a keyword.
- It must not contain white space.
- It should be up to 31 characters long as only the first 31 characters are significant.

Note: Identifiers are case-sensitive so names like *variable* and *Variable* will be treated as different.

For example,

- **main:** method name.
- **a:** variable name.

4. Constants

The constants refer to the variables with fixed values. They are like normal variables but with the difference that their values can not be modified in the program once they are defined.

Constants may belong to any of the data types.

Examples of Constants in C

```
const int c_var = 20;
```

```
const int* const ptr = &c_var;
```

5. Variables

In C programming language, a variable is a user-defined or a user-readable custom name assigned to a memory location. Variables hold a value that can be modified and reused many times during the program execution.

A variable can be an alphabet or digits and start with an underscore, but you cannot declare a keyword as a variable name.

C Variable Syntax

```
data_type variable_name = value; // defining single variable
```

or

```
data_type variable_name1, variable_name2; // defining multiple variable
```

Here,

- **data_type:** Type of data that a variable can store.
- **variable_name:** Name of the variable given by the user.
- **value:** value assigned to the variable by the user.

Example:

```
int a, b, sum;
```

6. Strings

Strings are nothing but an array of characters ended with a null character ('\0'). This null character indicates the end of the string. Strings are always enclosed in double quotes. Whereas a character is enclosed in single quotes in C and C++.

```
char str[50] = "Hello";
```

```
char str[] = "CWIT";
```

```
char str[] = { 'C','W','I','T'\0'};
```

7. Special Symbols

The following special symbols are used in C having some special meaning and thus, cannot be used for some other purpose. Some of these are listed below:

- **Brackets[]**: Opening and closing brackets are used as array element references. These indicate single and multidimensional subscripts.
- **Parentheses()**: These special symbols are used to indicate function calls and function parameters.
- **Braces{ }**: These opening and ending curly braces mark the start and end of a block of code containing more than one executable statement.
- **Comma (,)**: It is used to separate more than one statement like for separating parameters in function calls.
- **Colon(:)**: It is an operator that essentially invokes something called an initialization list.
- **Semicolon(;)**: It is known as a statement terminator. It indicates the end of one logical entity. That's why each individual statement must be ended with a semicolon.
- **Asterisk (*)**: It is used to create a pointer variable and for the multiplication of variables.
- **Assignment operator(=)**: It is used to assign values and for logical operation validation.
- **Pre-processor (#)**: The preprocessor is a macro processor that is used automatically by the compiler to transform your program before actual compilation.
- **Period (.)**: Used to access members of a structure or union.
- **Tilde(~)**: Used as a destructor to free some space from memory.

8. Operators

An operator in C can be defined as the symbol that helps us to perform some specific mathematical, relational, bitwise, conditional, or logical computations on values and variables. The values and variables used with operators are called operands. So we can say that the operators are the symbols that perform operations on operands.

Operators in C

	Operators	Type
Unary Operator →	++, --	Unary Operator
Binary Operator {	+, -, *, /, %	Arithmetic Operator
	<, <=, >, >=, ==, !=	Relational Operator
	&&, , !	Logical Operator
	&, , <<, >>, ~, ^	Bitwise Operator
	=, +=, -=, *=, /=, %=	Assignment Operator
Ternary Operator →	?:	Ternary or Conditional Operator

For example,

```
c = a + b;
```

Here, '+' is the operator known as the addition operator, and 'a' and 'b' are operands. The addition operator tells the compiler to add both of the operands 'a' and 'b'.

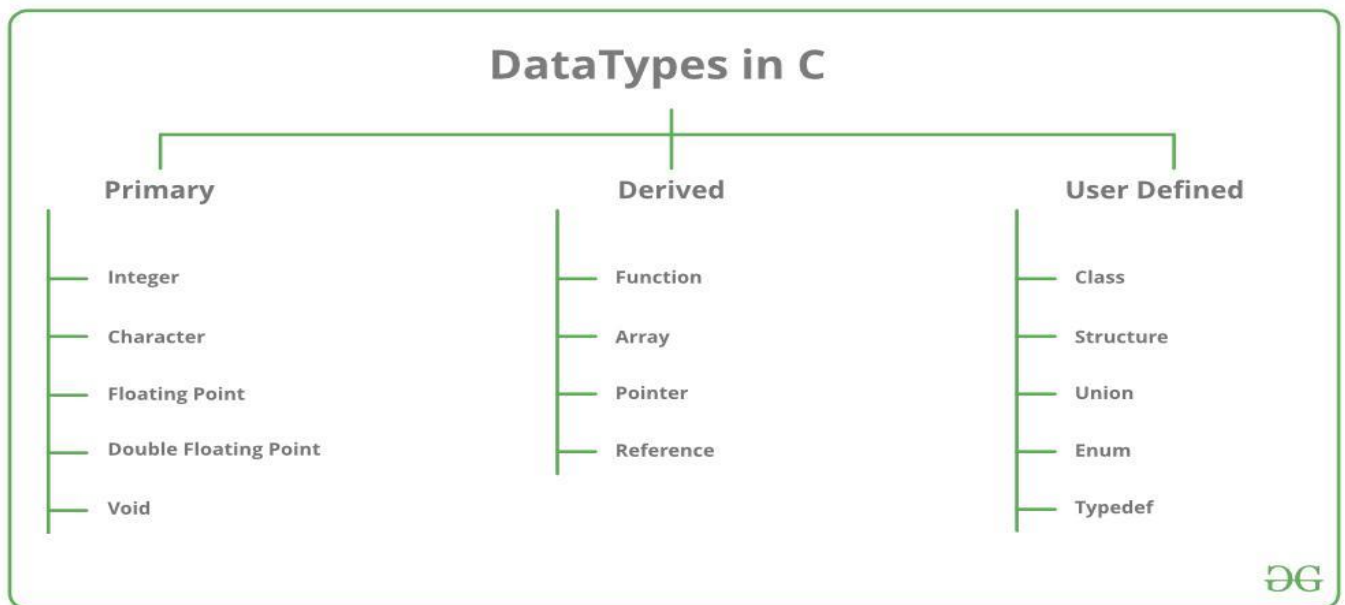
Types of Operators in C

C language provides a wide range of operators that can be classified into 6 types based on their functionality:

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Bitwise Operators
5. Assignment Operators
6. Other Operators

9. Data Types in C

Each variable in C has an associated data type. It specifies the type of data that the variable can store like integer, character, floating, double, etc. Each data type requires different amounts of memory and has some specific operations which can be performed over it. The data type is a collection of data with values having fixed values, meaning as well as its characteristics.



Different data types also have different ranges up to which they can store numbers. These ranges may vary from compiler to compiler. Below is a list of ranges along with the memory requirement and format specifiers on the **32-bit GCC compiler**.

Data Type	Size (bytes)	Range	Format Specifier
short int	2	-32,768 to 32,767	%hd

Data Type	Size (bytes)	Range	Format Specifier
unsigned short int	2	0 to 65,535	%hu
unsigned int	4	0 to 4,294,967,295	%u
int	4	-2,147,483,648 to 2,147,483,647	%d
long int	4	-2,147,483,648 to 2,147,483,647	%ld
unsigned long int	4	0 to 4,294,967,295	%lu
long long int	8	-(2 ⁶³) to (2 ⁶³)-1	%lld
unsigned long long int	8	0 to 18,446,744,073,709,551,615	%llu
signed char	1	-128 to 127	%c
unsigned char	1	0 to 255	%c

Data Type	Size (bytes)	Range	Format Specifier
float	4	1.2E-38 to 3.4E+38	%f
double	8	1.7E-308 to 1.7E+308	%lf
long double	16	3.4E-4932 to 1.1E+4932	%Lf

Note: The *long*, *short*, *signed* and *unsigned* are datatype modifier that can be used with some primitive data types to change the size or length of the datatype.

C Type Conversion

Sometimes, you have to convert the value of one data type to another type. This is known as **type conversion**.

For example, if you try to divide two integers, 5 by 2, you would expect the result to be 2.5. But since we are working with integers (and not floating-point values), the following example will just output 2:

Example

```
#include <stdio.h>

int main() {
    int x = 5;
    int y = 2;
    int sum = 5 / 2;
    printf("%d", sum);
}
```

```
    return 0;
}
```

Output : 2

To get the right result, you need to know how **type conversion** works.

There are two types of conversion in C:

- **Implicit Conversion** (automatically)
- **Explicit Conversion** (manually)

1. Implicit Conversion

Implicit conversion is done automatically by the compiler when you assign a value of one type to another.

For example, if you assign an int value to a float type:

```
#include <stdio.h>

int main() {
    // Automatic conversion: int to float
    float myFloat = 9;
    printf("%f", myFloat);
    return 0;
}
```

Output: 9.000000

2. Explicit Conversion

Explicit conversion is done manually by placing the type in parentheses () in front of the value.

Considering our problem from the example above, we can now get the right result:

Example

```
// Manual conversion: int to float
float sum = (float) 5 / 2;

printf("%f", sum);
```

Output

2.500000

1.4 Basic Input and Output functions:

Basic Input and Output in C

C language has standard libraries that allow input and output in a program. The **stdio.h** or **standard input output library** in C that has methods for input and output.

printf()

The printf() method, in C, prints the value passed as the parameter to it, on the console screen.

Syntax:

printf("%X", variableOfXType);

where %X is the format specifier in C. It is a way to tell the compiler what type of data is in a variable and & is the address operator in C, which tells the compiler to change the real value of this variable, stored at this address in the memory.

OR

Printf("Format string", list of variables or expression)

The format sting contain the following :

A] Character set

B] Blank Spaces

C] Escape sequences

D] Field width – The width of a value can be fixed by specifying the field width .

E] Format specifiers – Format specifiers specify the format of the variable data whose value is to be displayed.

scanf()

The scanf() method, in C, reads the value from the console as per the type specified.

Syntax:

Scanf(“format string” , address of variable);

The address of the variable is obtained with the help of the address of operator(&).

Example:

Scanf(“%d”,&x);

This statement is used to accept integer type value from user in the variable x.

Character input/Output statements:

This is unformatted I/O function.

The unformatted I/O function do not have any format specifies. They are mostly used to accept and display only one character or string.

Following are the unformatted I/O function:

getch(): The getch function is a way to get a single character from the user.

getche(): This function is used accept one char from the user and echo it. (Print or display)

getchar(): This function is used to accept one char from the user and echo it(display on the screen) and also wait after that for the user to press the enter key.

gets(): This function is used to accept a string from the user.

putch() or putchar(): These function are used to display a character on the monitor.

puts(): This function is used to display a string on the monitor.

1.5 Assignments and expressions:

1. Arithmetic Operations in C

The arithmetic operators are used to perform arithmetic/mathematical operations on operands. There are 9 arithmetic operators in C language:

S. No.	Symbol	Operator	Description	Syntax
1	+	Plus	Adds two numeric values.	a + b
2	-	Minus	Subtracts right operand from left operand.	a - b
3	*	Multiply	Multiply two numeric values.	a * b
4	/	Divide	Divide two numeric values.	a / b
5	%	Modulus	Returns the remainder after dividing the left operand with the right operand.	a % b
6	+	Unary Plus	Used to specify	+a

S. No.	Symbol	Operator	Description	Syntax
			the positive values.	
7	-	Unary Minus	Flips the sign of the value.	-a
8	++	Increment	Increases the value of the operand by 1.	a++
9	--	Decrement	Decreases the value of the operand by 1.	a--

Example of C Arithmetic Operators

// C program to illustrate the arithmetic operators
#include <stdio.h>

```
int main()
{
```

```
    int a = 25, b = 5;
```

```
    // using operators and printing results
```

```
    printf("a + b = %d\n", a + b);
    printf("a - b = %d\n", a - b);
    printf("a * b = %d\n", a * b);
    printf("a / b = %d\n", a / b);
    printf("a % b = %d\n", a % b);
    printf("+a = %d\n", +a);
    printf("-a = %d\n", -a);
    printf("a++ = %d\n", a++);
    printf("a-- = %d\n", a--);
```

```

    return 0;
}

```

Output

```

a + b = 30
a - b = 20
a * b = 125
a / b = 5
a % b = 0
+a = 25
-a = -25
a++ = 25
a-- = 26

```

2. Relational Operators in C

The relational operators in C are used for the comparison of the two operands. All these operators are binary operators that return true or false values as the result of comparison.

These are a total of 6 relational operators in C:

S. No.	Symbol	Operator	Description	Syntax
1	<	Less than	Returns true if the left operand is less than the right operand. Else false	a < b
2	>	Greater than	Returns true if the left operand is greater than the right operand. Else false	a > b
3	<=	Less than or equal to	Returns true if the left operand is less than or equal to	a <= b

S. No.	Symbol	Operator	Description	Syntax
			the right operand. Else false	
4	>=	Greater than or equal to	Returns true if the left operand is greater than or equal to right operand. Else false	a >= b
5	==	Equal to	Returns true if both the operands are equal.	a == b
6	!=	Not equal to	Returns true if both the operands are NOT equal.	a != b

Example of C Relational Operators

// C program to illustrate the relational operators

```
#include <stdio.h>
```

```
int main()
{
```

```
    int a = 25, b = 5;
```

```
    // using operators and printing results
```

```
    printf("a < b : %d\n", a < b);
```

```
    printf("a > b : %d\n", a > b);
```

```
    printf("a <= b: %d\n", a <= b);
```

```
    printf("a >= b: %d\n", a >= b);
```

```
    printf("a == b: %d\n", a == b);
```

```
    printf("a != b : %d\n", a != b);
```

```
    return 0;
```

```
}
```

Output

a < b : 0

a > b : 1

a <= b: 0

a >= b: 1

a == b: 0

a != b : 1

Here, 0 means false and 1 means true.

3. Logical Operator in C

Logical Operators are used to combine two or more conditions/constraints or to complement the evaluation of the original condition in consideration. The result of the operation of a logical operator is a Boolean value either **true** or **false**.

S. No.	Symbol	Operator	Description	Syntax
1	&&	Logical AND	Returns true if both the operands are true.	a && b
2		Logical OR	Returns true if both or any of the operand is true.	a b
3	!	Logical NOT	Returns true if the operand is false.	!a

Example of Logical Operators in C

```
// C program to illustrate the logical operators
#include <stdio.h>
```

```
int main()
{
    int a = 25, b = 5;

    // using operators and printing results
    printf("a && b : %d\n", a && b);
    printf("a || b : %d\n", a || b);
    printf("!a: %d\n", !a);

    return 0;
}
```

Output:

a && b : 1

a || b : 1

!a: 0

4. Bitwise Operators in C

The Bitwise operators are used to perform bit-level operations on the operands. The operators are first converted to bit-level and then the calculation is performed on the operands. Mathematical operations such as addition, subtraction, multiplication, etc. can be performed at the bit level for faster processing.

There are 6 bitwise operators in C:

S. No.	Symbol	Operator	Description	Syntax
1	&	Bitwise AND	Performs bit-by-bit AND operation and returns the result.	a && b
2		Bitwise OR	Performs bit-by-bit OR operation	a b

S. No.	Symbol	Operator	Description	Syntax
3	$\wedge$	Bitwise XOR	and returns the result. Performs bit-by-bit XOR operation and returns the result.	$a \wedge b$
4	$\sim$	Bitwise First Complement	Flips all the set and unset bits on the number.	$\sim a$
5	$\ll$	Bitwise Leftshift	Shifts the number in binary form by one place in the operation and returns the result.	$a \ll b$
6	$\gg$	Bitwise Rightshilft	Shifts the number in binary form by one place in the operation and returns the result.	$a \gg b$

Example of Bitwise Operators

// C program to illustrate the bitwise operators
#include <stdio.h>

```
int main()
{
    int a = 25, b = 5;
```

```
    // using operators and printing results
    printf("a & b: %d\n", a & b);
```

```

printf("a | b: %d\n", a | b);
printf("a ^ b: %d\n", a ^ b);
printf("~a: %d\n", ~a);
printf("a >> b: %d\n", a >> b);
printf("a << b: %d\n", a << b);

return 0;
}

```

Output

a & b: 1

a | b: 29

a ^ b: 28

~a: -26

a >> b: 0

a << b: 800

5. Assignment Operators in C

Assignment operators are used to assign value to a variable. The left side operand of the assignment operator is a variable and the right side operand of the assignment operator is a value. The value on the right side must be of the same data type as the variable on the left side otherwise the compiler will raise an error.

The assignment operators can be combined with some other operators in C to provide multiple operations using single operator. These operators are called compound operators.

In C, there are 11 assignment operators :

S. No.	Symbol	Operator	Description	Syntax
1	=	Simple Assignment	Assign the value of the right operand to the left operand.	a = b
2	+=	Plus and assign	Add the right operand and left operand and assign this value to the left operand.	a += b

S. No.	Symbol	Operator	Description	Syntax
3	-=	Minus and assign	Subtract the right operand and left operand and assign this value to the left operand.	a -= b
4	*=	Multiply and assign	Multiply the right operand and left operand and assign this value to the left operand.	a *= b
5	/=	Divide and assign	Divide the left operand with the right operand and assign this value to the left operand.	a /= b
6	%=	Modulus and assign	Assign the remainder in the division of left operand with the right operand to the left operand.	a %= b
7	&=	AND and assign	Performs bitwise AND and assigns this value to the left operand.	a &= b
8	 =	OR and assign	Performs bitwise OR and assigns this value to the left operand.	a = b
9	^=	XOR and assign	Performs bitwise XOR and assigns	a ^= b

S. No.	Symbol	Operator	Description	Syntax
			this value to the left operand.	
10	>>=	Rightshift and assign	Performs bitwise Rightshift and assign this value to the left operand.	a >>= b
11	<<=	Leftshift and assign	Performs bitwise Leftshift and assign this value to the left operand.	a <<= b

Example of C Assignment Operators

// C program to illustrate the arithmetic operators

#include <stdio.h>

```
int main()
{
```

```
    int a = 25, b = 5;
```

```
    // using operators and printing results
```

```
    printf("a = b: %d\n", a = b);
```

```
    printf("a += b: %d\n", a += b);
```

```
    printf("a -= b: %d\n", a -= b);
```

```
    printf("a *= b: %d\n", a *= b);
```

```
    printf("a /= b: %d\n", a /= b);
```

```
    printf("a %= b: %d\n", a %= b);
```

```
    printf("a &= b: %d\n", a &= b);
```

```
    printf("a |= b: %d\n", a |= b);
```

```
    printf("a >>= b: %d\n", a >> b);
```

```
    printf("a <<= b: %d\n", a << b);
```

```
    return 0;
```

```
}
```

Output

a = b: 5

a += b: 10

a -= b: 5

a *= b: 25

a /= b: 5

a %= b: 0

a &= b: 0

a |= b: 5

)a >>= b: 0

a <<= b: 160

Other Operators

Apart from the above operators, there are some other operators available in C used to perform some specific tasks. Some of them are discussed here:

6. sizeof Operator

- sizeof is much used in the C programming language.
- It is a compile-time unary operator which can be used to compute the size of its operand.
- The result of sizeof is of the unsigned integral type which is usually denoted by size_t.
- Basically, the sizeof the operator is used to compute the size of the variable or datatype.

Syntax

sizeof (operand)

I. Comma Operator (,)

- The comma operator (represented by the token) is a binary operator that evaluates its first operand and discards the result, it then evaluates the second operand and returns this value (and type).
- The comma operator has the lowest precedence of any C operator.
- Comma acts as both operator and separator.

Syntax

- operand1 , operand2

II. Conditional Operator (? :)

- The conditional operator is the only ternary operator in C++.
- Here, Expression1 is the condition to be evaluated. If the condition(Expression1) is *True* then we will execute and return the result of Expression2 otherwise if the condition(Expression1) is *false* then we will execute and return the result of Expression3.
- We may replace the use of if..else statements with conditional operators.

Syntax

- operand1 ? operand2 : operand3;

III. dot (.) and arrow (->) Operators

- Member operators are used to reference individual members of classes, structures, and unions.
- The dot operator is applied to the actual object.
- The arrow operator is used with a pointer to an object.

Syntax

structure_variable . member;
and
structure_pointer -> member;

IV. Cast Operator

- Casting operators convert one data type to another. For example, int(2.2000) would return 2.
- A cast is a special operator that forces one data type to be converted into another.
- The most general cast supported by most of the C compilers is as follows – [**(type) expression**].

Syntax

(new_type) operand;

V. addressof (&) and Dereference (*) Operators

- Pointer operator & returns the address of a variable. For example &a; will give the actual address of the variable.
- The pointer operator * is a pointer to a variable. For example *var; will pointer to a variable var.

Example of Other C Operators

// C Program to demonstrate the use of Misc operators

```
#include <stdio.h>
```

```
int main()
{
    // integer variable
    int num = 10;
    int* add_of_num = #

    printf("sizeof(num) = %d bytes\n", sizeof(num));
    printf("&num = %p\n", &num);
    printf("*add_of_num = %d\n", *add_of_num);
    printf("(10 < 5) ? 10 : 20 = %d\n", (10 < 5) ? 10 : 20);
    printf("(float)num = %f\n", (float)num);

    return 0;
}
```

Output

sizeof(num) = 4 bytes

&num = 0x7ffe2b7bdf8c

*add_of_num = 10

(10 < 5) ? 10 : 20 = 20

(float)num = 10.000000

7. Shift Operators

I. Left Shift(<<)

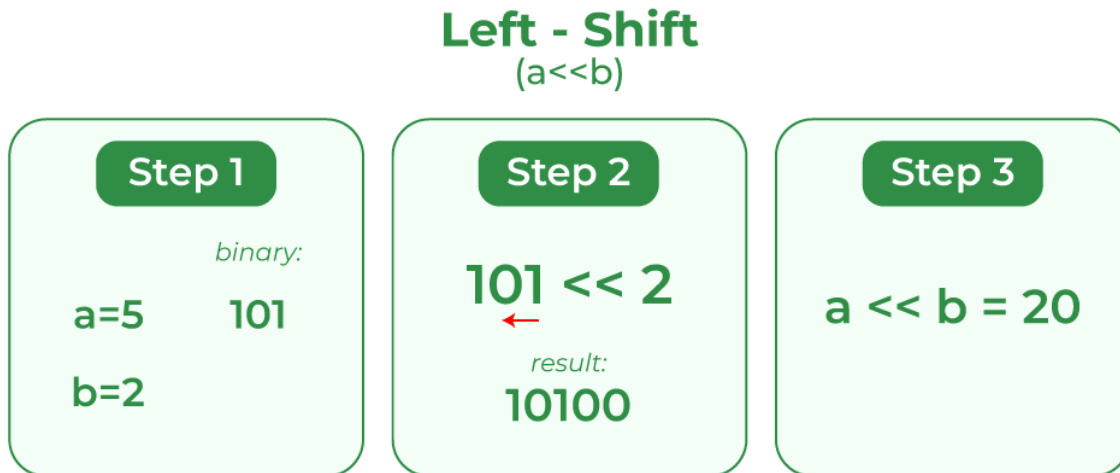
It is a binary operator that takes two numbers, left shifts the bits of the first operand, and the second operand decides the number of places to shift. In other words, left-shifting an integer "a" with an integer "b" denoted as '(a<<b)' is equivalent to multiplying a with 2^b (2 raised to power b).

Syntax:

a << b;

- a: First Operand
- b: Second Operand

Example: Let's take $a=5$; which is 101 in Binary Form. Now, if "*a is left-shifted by 2*" i.e $a=a<<2$ then a will become $a=a*(2^2)$. Thus, $a=5*(2^2)=20$ which can be written as 10100



// C Program to demonstrate use of left shift operator

```
#include <stdio.h>
```

```
// Driver code
```

```
int main()
```

```
{
```

```
    // a = 5(00000101), b = 9(00001001)
```

```
    unsigned char a = 5, b = 9;
```

```
    // The result is 00001010
```

```
    printf("a<<1 = %d\n", (a << 1));
```

```
    // The result is 00010010
```

```
    printf("b<<1 = %d", (b << 1));
```

```
    return 0;
```

```
}
```

Output

```
a<<1 = 10
```

```
b<<1 = 18
```

II. Right Shift(>>)

It is a binary operator that takes two numbers, right shifts the bits of the first operand, and the second operand decides the number of places to shift. In other words, right-shifting an integer "*a*" with an integer "*b*" denoted as ' $(a>>b)$ ' is equivalent to dividing *a* with 2^b .

Syntax:

`a >> b;`

- a: First Operand
- b: Second Operand

Example: let's take `a=5`; which is 101 in Binary Form. Now, if "*a is right-shifted by 2*" i.e `a=a>>2` then a will become $a=a/(2^2)$. Thus, $a=a/(2^2)=1$ which can be written as 01.

Right - Shift (`a>>b`)

**// C Program to demonstrate use of right-shift operator**

```
#include <stdio.h>
```

```
// Driver code
```

```
int main()
```

```
{
```

```
    // a = 5(00000101), b = 9(00001001)
```

```
    unsigned char a = 5, b = 9;
```

```
    // The result is 00000010
```

```
    printf("a>>1 = %d\n", (a >> 1));
```

```
    // The result is 00000100
```

```
    printf("b>>1 = %d", (b >> 1));
```

```
    return 0;
```

```
}
```

Output

```
a>>1 = 2
```

```
b>>1 = 4
```

Important Points to remember about left-shift & right-shift operator

1. The left-shift and right-shift operators should not be used for negative numbers. The result of is undefined behaviour if any of the operands is a negative number. For example results of both $1 \gg -1$ and $1 \ll -1$ is undefined.
2. If the number is shifted more than the size of the integer, the behaviour is undefined. For example, $1 \ll 33$ is undefined if integers are stored using 32 bits. For bit shift of larger values $1ULL \ll 62$ **ULL** is used for Unsigned Long Long which is defined using 64 bits that can store large values.
3. The left-shift by 1 and right-shift by 1 are equivalent to the product of the first term and 2 to the power given element ($1 \ll 3 = 1 * \text{pow}(2,3)$) and division of the first term and second term raised to power 2 ($1 \gg 3 = 1 / \text{pow}(2,3)$) respectively.

Operator Precedence and Associativity in C

- In C, it is very common for an expression or statement to have multiple operators and in these expression, there should be a fixed order or priority of operator evaluation to avoid ambiguity.
- **Operator Precedence and Associativity** is the concept that decides which operator will be evaluated first in the case when there are multiple operators present in an expression.

Precedence	Operator	Description	Associativity
1	()	Parentheses (function call)	left-to-right
	[]	Brackets (array subscript)	left-to-right
	.	Member selection via object name	left-to-right
	->	Member selection via a pointer	left-to-right
	a++ , a--	Postfix increment/decrement (a is a variable)	left-to-right

Precedence	Operator	Description	Associativity
2	++a , -a	Prefix increment/decrement (a is a variable)	right-to-left
	+ , -	Unary plus/minus	right-to-left
	! , ~	Logical negation/bitwise complement	right-to-left
	(type)	Cast (convert value to temporary value of type)	right-to-left
	*	Dereference	right-to-left
	&	Address (of operand)	right-to-left
3	sizeof	Determine size in bytes on this implementation	right-to-left
3	*, / , %	Multiplication/division/modulus	left-to-right
4	+ , -	Addition/subtraction	left-to-right
5	<< , >>	Bitwise shift left, Bitwise shift right	left-to-right
6	< , <=	Relational less than/less than or equal to	left-to-right
	> , >=	Relational greater than/greater than or equal to	left-to-right
	== , !=	Relational is equal to/is not equal to	left-to-right
	&	Bitwise AND	left-to-right

Precedence	Operator	Description	Associativity
9	^	Bitwise exclusive OR	left-to-right
10	 	Bitwise inclusive OR	left-to-right
11	&&	Logical AND	left-to-right
12	 	Logical OR	left-to-right
13	?:	Ternary conditional	right-to-left
	=	Assignment	right-to-left
	+= , -=	Addition/subtraction assignment	right-to-left
	*= , /=	Multiplication/division assignment	right-to-left
	%= , &=	Modulus/bitwise AND assignment	right-to-left
	^= , =	Bitwise exclusive/inclusive OR assignment	right-to-left
14	<<= , >>=	Bitwise shift left/right assignment	right-to-left
15	,	expression separator	left-to-right